Fleet Mucky Mayhem

A beginner’s guide to the website code

The browser displays the website

The browser is the app showing the page, such as Edge or Chrome.

Website files describe what to show and how it behaves.

An editor lets us change those files.

The browser view and the source files are two views of the same work.

Each kind of code has a job

HTML describes content and structure: a heading, paragraph or link.

CSS controls appearance: colour, spacing and layout.

JavaScript handles behaviour: clicks, menus and changing content.

JSON stores information: ride details and photo links.

Reading a little HTML

```
<h2 class="brand-text display-2">
  Contact Us
</h2>
```

<h2> starts a heading.

Contact Us is the visible text.

</h2> ends the heading.

class names connect this element to styling rules.

Heading from ContactUs.astro, line breaks added for reading

The file endings are clues

.astro A page, a section or the shared layout

.scss Styling rules, using the Sass format

.js / .ts Behaviour and helper functions

.json Structured information

.jpg / .png Image files

The folders you will use

```
fleet-mucky-mayhem/
  src/pages/
  src/layouts/
  src/components/
  src/scss/
  src/js/ and src/scripts/
  src/lib/
  public/data/
  public/img/
```

pages and layouts assemble the page.

components holds the visible sections.

scss, scripts and lib control appearance and behaviour.

public holds data files and images.

All paths start inside the fleet-mucky-mayhem project folder

The homepage sets the section order

```
<SiteLayout>
  <Navigation />
  <Hero />
  <AboutUs />
  <WhatWeDo />
  <WhereToNext />
  ...
  <Footer />
</SiteLayout>
```

index.astro is the homepage file.

Each named component adds a section.

The order here matches the page order.

The import lines tell Astro where those components live.

Shortened excerpt from src/pages/index.astro; ... marks omitted sections

The layout wraps the sections

```
<body data-bs-theme="dark">
  <slot />
  <script src="../scripts/site.ts"></script>
</body>
```

SiteLayout.astro supplies the shared page wrapper.

<slot /> marks where the homepage sections go.

The layout also sets the browser title and loads styles.

site.ts brings in the main browser interactions.

Exact body excerpt from src/layouts/SiteLayout.astro

The component names match the page

Hero.astro contains the opening welcome text.

AboutUs.astro contains the About Us content and carousel.

RideDiary.astro contains regular rides and space for longer rides.

Faq.astro and ContactUs.astro contain those sections.

Photos.astro works with photo data and a browser script.

A small text edit

Before

```
<p>
  If you join, please introduce yourself to the group.
</p>
```

After

```
<p>
  Please introduce yourself when you join the group.
</p>
```

The words between and form a paragraph.

Changing those words changes the message.

The surrounding tags keep the structure.

This is a practice example, not an agreed client copy change.

Original paragraph from src/components/ContactUs.astro

Links, IDs and classes

```
<a href="#contact-us">Contact Us</a>

<section id="contact-us"
         class="even-sections">
```

href is the destination of a link.

#contact-us points to an ID on this page.

An ID names one specific element.

A class is a reusable label, often used for styling.

Shortened excerpts from Hero.astro and ContactUs.astro

Images have a file and a description

```
<img src="img/mm-qr-code.jpg"
     class="img-fluid contact-us-qr-code"
     alt="QR Code for WhatsApp Group">
```

src points to the image file.

This image lives in public/img/mm-qr-code.jpg.

alt describes the image for people who cannot see it.

Matching filenames and letter case prevents broken images.

Exact image element, rewrapped, from ContactUs.astro

A CSS rule changes appearance

```
.brand-text {
  font-family: "Urbanist", sans;
  font-style: italic;
  font-weight: 600;
  color: var(--bs-warning);
}
```

.brand-text selects elements with that class.

Inside { } are the appearance settings.

Each setting has a name, a colon and a value.

--bs-warning supplies Bootstrap’s warning colour, used as the brand yellow.

Exact rule from src/scss/styles.scss

Bootstrap supplies common styles

```
class="btn btn-lg btn-warning"

class="container"

class="row"
class="col"
```

Bootstrap is a collection of ready-made styles and behaviours.

btn styles a button. btn-lg makes it larger.

container, row and col help arrange content.

Responsive rules help the layout adapt to screen width.

Class examples used across the Astro components

Styles can apply at different scales

src/scss/styles.scss loads shared styles and contains global rules.

src/scss/base/base.scss sets defaults such as section spacing.

A <style> block in an Astro component usually targets that component.

Media queries apply rules at certain screen widths.

A shared style change needs a check across several sections.

Pause and check the file choices

A contact sentence needs changing. Which file?

A shared yellow heading style needs changing. Which file?

The opening section order needs changing. Which file?

An image is missing. What two things would you compare?

JSON keeps information in named fields

```
{
  "title": "Canal Café",
  "distanceMiles": 30,
  "date": null,
  "notes": []
}
```

A field has a name and a value.

Text uses double quotes. Numbers do not.

null means no date is set in this example.

[ ] holds a list. An empty list is [].

Teaching example only: a fragment, not a complete ride record

One ride is one record

```
"id": "war-of-the-worlds-2026-03-14",
"title": "War of the Worlds",
"date": "2026-03-14",
"year": 2026,
"startTime": "09:30",
"destination": "Woking",
"distanceMiles": 36,
```

These fields belong to one archived ride.

id gives the record a unique identity.

date uses year-month-day. startTime uses a 24-hour clock.

The complete record also includes meeting details and other information.

Selected fields from public/data/longer-rides.json; surrounding record omitted

The Ride Diary reads the ride data

1. public/data/longer-rides.json stores the records.

2. src/lib/data.ts loads and checks those records.

3. src/js/ride-diary.js creates the longer ride entries.

4. RideDiary.astro supplies the area where they appear.

Regular ride wording lives directly in RideDiary.astro.

Where To Next reuses the ride data

The page selects the earliest future departure from longerRides.

A ride needs a date to become the next ride.

Departure times use the UK time zone, including clock changes.

Archived rides never become upcoming rides.

With no future ride, this version shows its Midwinter Celebration fallback.

Photos has its own data file

```
{
  "year": 2026,
  "albums": [
    {
      "label": "14 Mar : War of the Worlds",
      "url": "https://app.box.com/s/..."
    }
  ]
}
```

photos.json groups album links by year.

The visitor’s UK calendar year selects current and archive menus.

A ride’s photosUrl is a separate link.

If an album URL appears in both places, update both.

Shortened photos.json example; URL abbreviated, not a link to copy

The ride template needs real values

ride-template.json is a starting example, not the live ride list.

Copy only newRide into longer-rides.json, then complete it.

Use a unique ID and a real date, or null for an undecided date.

Remove unused optional fields, including an empty meeting-point note.

Replace placeholder links or remove optional links until they are ready.

JavaScript responds to events

An element is one item on the page, such as a button.

A variable gives a name to a value.

A function groups instructions under a name.

An event is something that happens, such as a click.

A script can find an element and attach a function to its click event.

A click changes the active menu item

```
function setActiveLink(link) {
  navLinkEls.forEach(item => {
    item.classList.toggle('active', item === link);
  });
}
```

Use only the links selected inside the navigation.

Compare each item with the clicked link.

Keep active on the matching link only.

Leave carousel and other controls independent.

Shortened helper from src/js/navlink.js; aria-current lines omitted

Bootstrap connects a button to hidden content

```
<button data-bs-toggle="collapse"
        data-bs-target="#question-one">
  1. When, Where and What time does the group meet up?
</button>

<div id="question-one"
     class="accordion-collapse collapse">
```

data-bs-toggle names the Bootstrap behaviour.

data-bs-target names the element to open or close.

The target matches that element’s ID.

Bootstrap provides the expand-and-collapse behaviour.

Shortened excerpt from src/components/Faq.astro; styling and other attributes omitted

A small investigation

Wrong words? Find the exact phrase in the source.

Missing image? Compare its path with the real filename.

Wrong link? Check href or the relevant JSON URL.

A control does nothing? Check its ID and target, then the browser console.

After a change, check the affected area at wide and narrow widths.

The file choices to remember

Words in a section: src/components/

Longer ride details: public/data/longer-rides.json

Photo menu links: public/data/photos.json

Appearance: src/scss/ or a component’s <style> block

Clicks and changing content: src/js/, src/scripts/ and helpers in src/lib/